README | Webots PY

Proyecto wbtlab11_Py: seguimiento circular de cuatro robots Pololu 3Pi en Webots con controladores Python.

Estructura del proyecto

Ruta	Contenido
worlds/Trayectory_4agents.wbt	Único mundo incluido; declara el formato R2025a.
controllers/robot1/robot1.py	Líder con trayectoria circular y selección discreta de objetivos.
controllers/robot2/robot2.py	Sigue a ROBOT1 mediante la API Supervisor.
controllers/robot3/robot3.py	Sigue a ROBOT2 mediante la API Supervisor.
controllers/robot4/robot4.py	Sigue a ROBOT3 mediante la API Supervisor.
Modelos_3D/Pololu/ y protos/	Mallas STL y modelo Pololu3pi.proto.
Otros controladores y robotat/	Auxiliares MATLAB heredados; no están activos en este mundo.

Requisitos y ejecución

1. Extraer el ZIP completo conservando las carpetas. Usar Webots con un intérprete Python compatible configurado. Los controladores activos importan math y el módulo controller proporcionado por Webots; no usan NumPy ni requieren MATLAB.

2. En el mundo, sustituir las rutas absolutas a BumperV2.stl por la ruta relativa al modelo incluido:

```
../Modelos_3D/Pololu/BumperV2.stl
```

3. Abrir worlds/Trayectory_4agents.wbt y ejecutar la simulación desde Webots. Los scripts se inician como controladores de los nodos Robot, no como programas independientes en una terminal.

4. Mantener estas asignaciones. Los nombres de controlador corresponden a sus carpetas:

Nodo DEF	Controlador / archivo	Supervisor
ROBOT1	robot1 / robot1.py	No activado
ROBOT2	robot2 / robot2.py	TRUE
ROBOT3	robot3 / robot3.py	TRUE
ROBOT4	robot4 / robot4.py	TRUE

Los seguidores necesitan supervisor TRUE para consultar al agente precedente. Cada robot utiliza sus propios dispositivos motor_1, motor_2, gps y compass. El mundo define los robots directamente y no instancia Pololu3pi.proto.

Resultado

El líder recorre objetivos de un círculo y los seguidores forman una cadena. El movimiento se observa en Webots; no se generan gráficas, historiales ni archivos de resultados automáticamente.

Trayectoria y control

Parámetros principales

Parámetro	Líder: robot1.py	Seguidores: robot2-4.py
TIME_STEP	64 ms	64 ms
Ra / NV / tf	600 mm / 5 / 40 s	No aplica
v0 / alpha	150 mm/s / 0.0001	200 mm/s / 0.0001
kpO / kiO / kdO	8 / 0.0001 / 0.2	6 / 0.0001 / 0.1
Anticipación / tolerancia	10 muestras / 100 mm	No aplica
d_sep / eP_stop	No aplica	300 mm / 30 mm

Líder: robot1.py

Crea un objeto Robot y espera lecturas válidas de GPS y Compass. El centro del círculo se coloca a 600 mm a la izquierda de la orientación inicial. Calcula 626 puntos con dt = 0.064 s y w_traj = NV*2*pi/tf.

```
ang = ang0 + w_traj*tk
xg  = xc + Ra*math.cos(ang)
yg  = yc + Ra*math.sin(ang)
```

El índice inicia en cero. Cuando eP <= 100 mm, avanza 10 muestras hasta N_pts - 1. El error ya calculado controla ese paso; el nuevo objetivo se usa en el siguiente. tf define la trayectoria, pero no detiene Webots tras 40 s ni garantiza completar las cinco vueltas.

Seguidores: robot2.py, robot3.py y robot4.py

Crean un objeto Supervisor. LEADER_DEF identifica al agente precedente: ROBOT1, ROBOT2 y ROBOT3, respectivamente. El objetivo se ubica sobre la línea que une ambos robots, a d_sep del precedente:

```
dx = lx - posx
dy = ly - posy
dist = math.hypot(dx, dy)
xg = lx - d_sep*dx/dist   # si dist > 1e-3
yg = ly - d_sep*dy/dist
```

Si dist <= 1e-3, la dirección se toma como cero. Si eP < 30 mm, se anulan las velocidades y se reinician EO y eO_1. El objetivo depende de la posición del precedente, no de su orientación.

Ley de control compartida

eP es la distancia al objetivo. eO es el error angular normalizado a través de atan2(sin(error),cos(error)). Fuera de la zona de parada de los seguidores:

```
v = v0*(1 - math.exp(-alpha*eP**2))
eO_D = eO - eO_1
EO = EO + eO
w = kpO*eO + kiO*EO + kdO*eO_D
eO_1 = eO
```

La posición GPS se convierte de metros a milímetros; v se expresa en mm/s y w en rad/s. La suma integral y la diferencia derivativa no se escalan con dt. Las ganancias deben revisarse si cambia el período de control.

Motores, configuración y auxiliares

Conversión a velocidades de rueda

Los controladores usan radio de rueda r = 32 mm y separación L = 96 - 2*6.8 = 82.4 mm. A partir de v y w se calculan velocidades de rueda en rad/s:

```
q_right = (2*v + w*L)/(2*r)
q_left  = (2*v - w*L)/(2*r)
rpm     = q*(60/(2*math.pi))
```

Cada rueda se limita a +/-400 rpm mediante velsat. Después se convierte de nuevo a rad/s y se aplica el signo del montaje del robot en Webots:

```
left_motor.setVelocity(-rpm2rads(v_left))
right_motor.setVelocity(-rpm2rads(v_right))
```

El movimiento lo calcula la física de Webots a partir de los motores. Estos scripts Python no integran explícitamente el uniciclo mediante RK4 ni escriben las poses de los robots.

Detalles de configuración

Orientación: los cuatro scripts usan math.atan2(mag[1],mag[0]) - 90.0. atan2 devuelve radianes. Si se pretende un desfase de 90 grados, debe revisarse con math.pi/2 y comprobar el signo y los ejes de la escena.

Sensores y tiempo: GPS y Compass se habilitan a 64 ms. Verificar que TIME_STEP sea coherente con el basicTimeStep del mundo. Los ciclos principales continúan mientras robot.step(TIME_STEP) != -1.

Formación: d_sep es la distancia al objetivo sobre la línea entre robots; no impone una separación exacta durante el movimiento. No hay prevención de colisiones ni anti-windup general. Los seguidores reinician su integrador dentro de la zona de parada.

Archivos MATLAB incluidos

Estos archivos permanecen en el ZIP, pero el mundo no los ejecuta y no son conversiones a Python:

Archivo o carpeta	Función heredada
central_controller.m	Alternativa centralizada: motores de robot1 y poses de robots 2-4. Requiere MATLAB y reasignar los controladores del mundo.
pololu3pi_controller.m	Control individual con modos punto a punto y trayectoria quintica; llama a quinticpolytraj.
robot_receiver.m	Recibe comandos de ruedas en rpm. Requiere receiver y un emisor; el mundo incluido no los incorpora.
pololu3pi_controller/robotat/	Conexiones TCP, poses, comandos de robots físicos y utilidades de cuaterniones en MATLAB.

Modelos y recursos

Pololu3pi.proto conserva my_controller_EpuckPruebas, ausente del ZIP; revisar esa asignación si se utiliza el PROTO en otro mundo. Las referencias EXTERNPROTO del mundo dependen de recursos externos de Webots. La simulación activa no abre conexiones a Robotat ni necesita un robot físico.